

Projeto e Análise de Algoritmos I

Aula 01 - Introdução

Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul
Instituto de Informática
Departamento de Informática Teórica



Estes slides utilizam conteúdo adaptado da bibliografia da disciplina e também de notas de aula e slides prévios dos professores Bruno Grisci, Rodrigo Machado e André Grahl.

Apresentação da Disciplina

Dados Gerais

- **Disciplina:** INF05027 - Projeto e Análise de Algoritmos I
- **Professor:** Lucas Nunes Alegre
- **Carga horária:** 60h (50 em sala de aula, 10 em atividades autônomas)
- **Créditos:** 4

Sobre o Professor

Lucas Nunes Alegre:

- **E-mail:** lnalegre@inf.ufrgs.br
- **Site:** <https://lucasalegre.github.io/>
- **Presencial:** Sala 233, prédio 43424 (segundo andar)

Interesses:

- Aprendizado por Reforço.
- Inteligência Artificial.
- Aprendizado de Máquina
- Robótica.
- Análise de Redes.



Motivação

As mudanças no currículo vão na seguinte direção:

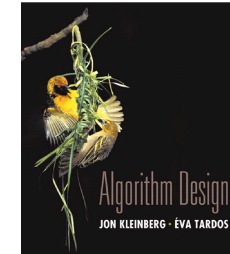
- Aprofundar a forma de se estudar algoritmos (sempre com análise de **correção** e **custos**)
- Integração natural de **teoria** e **prática**
- Reorganizar a exposição de tópicos de forma a seguir as referências da literatura mundial da área

Desta forma, *PAA1* + *PAA2* no novo currículo cobrem conteúdos que eram vistos de forma não integrada em: - **Teoria dos Grafos e Análise Combinatória** - **Complexidade de Algoritmos** - **Classificação e Pesquisa de Dados**

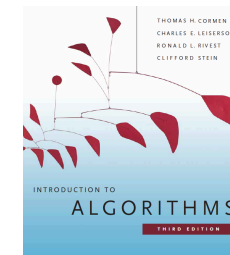
Bibliografia

Bibliografia:

- **Algorithm Design.** Kleinberg, Jon; Tardos, Éva.
- **Algoritmos: Teoria e Prática.** Cormen, Thomas H. et. al.
- **Algorithms Illuminated (Parts 1, 2, 3).** Tim Roughgarden.



KT

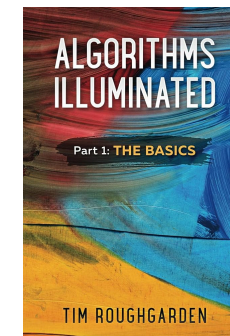


Cor

Moodle:

<https://moodle.ufrgs.br/>

- Toda a comunicação oficial.



AI

Página da Disciplina

<https://lucasalegre.github.io/paa1>

Repositório - Github

GitHub:

<https://github.com/BrunoGrisci/projeto-e-analise-de-algoritmos>

- Diversidade de autores e linguagens de programação.
- Disponibilização de códigos, demos, exemplos.

Avaliação

Atividades Avaliadas:

- **Prova 1 (3.5pts)** - Análise assintótica, teoria dos grafos, algoritmos simples sobre grafos
- **Prova 2 (3.5pts)** - Algoritmos gulosos, algoritmos gulosos sobre grafos, estruturas de dados avançadas
- **Laboratórios (2pt)**
- Atividade(s) autônoma(s) (1pt)

$$NF = 0.35 \times P1 + 0.35 \times P2 + 0.2 \times Lab + 0.1 \times Aut$$

Conceito definido a partir de NF da forma tradicional: A (≥ 9), B (≥ 7.5), C (≥ 6), D (< 6), FF (menos de 75% de frequência).

Laboratórios

- 4 laboratórios ao longo do semestre
- Atividades avaliadas (0.5 pontos cada)
- Entrega individual, mas discussão e consulta liberadas
- **Presencial!**
- Implementação de algoritmos/análise na prática
- Ambientes de desafios
 - LeetCode: <https://leetcode.com/>
 - beecrowd: <https://beecrowd.com/>
 - . . .
- Linguagem de programação livre (dentro dos limites das ferramentas)
- Recomendação: **Python**

Python

- De C para Python:
https://colab.research.google.com/drive/1NTX2_Xdrp_uBE72-Wmnld2fL5lTmG1Ha?usp=sharing
- É **fortemente** sugerido aprender Python para esta disciplina.
 - Algoritmos em aula serão apresentados em **pseudocódigo** e/ou **Python**
 - “Mas eu não gosto de Python...” - 2ª opção: **C++**
- Programar é como tocar um instrumento: é necessário **praticar toda semana!**
- Exercícios e materiais de programação sugeridos na página da disciplina:
Programação

Atividade de Recuperação

Exame de Recuperação

- Prova sobre todo o conteúdo
- Final do semestre

Nova nota final recalculada seguindo a seguinte fórmula:

$$NE = 0.2 \times NF + 0.8 \times EXAME$$

Conteúdo

Análise de Algoritmos

- Correção de algoritmos
- Análise assintótica do custo computacional de algoritmos

Grafos: conceitos e algoritmos fundamentais

- Grafos: definição e representações computacionais
- Principais conceitos e algoritmos sobre grafos: busca, componentes, distância.

Estratégia gulosa de projeto de algoritmos:

- Escalonamento e particionamento de intervalos

Algoritmos gulosos para grafos e estruturas de dados avançadas:

- Árvore geradora de custo mínimo (Prim, Kruskal, union-find)
- Distância em grafos com pesos (Dijkstra, priority queues)

Contexto no Currículo

Antes:

- INF01202 - ALGORITMOS E PROGRAMAÇÃO - CIC
- INF05008 - PENSAMENTO COMPUTACIONAL N
- INF01203 - ESTRUTURAS DE DADOS
- MAT01375 - MATEMÁTICA DISCRETA B

Depois:

- INF05028 - PROJETO E ANÁLISE DE ALGORITMOS II
- INF05501 - TEORIA DA COMPUTAÇÃO II
- INF01048 - INTELIGÊNCIA ARTIFICIAL
- INF05010 - OTIMIZAÇÃO COMBINATÓRIA

Na verdade

- O estudo dos algoritmos permeia a totalidade da ciência da computação e áreas relacionadas.

Motivação: Programação Competitiva

Laboratório de Programação Competitiva - LPC:

- <https://www.inf.ufrgs.br/maratonas/>
- <https://www.instagram.com/lpcufrgs>

MARATONA DE PROGRAMAÇÃO SBC

APOIO: appmax widelabs TELUS Digital APRIX

Nelogica infrati MBO CHIP DELL Int6 tech

lpcufrgs
INF - UFRGS

lpcufrgs No último sábado, o LPC organizou, junto com os professores Rodrigo Machado e Henrique Becker, a Fase Regional da Maratona de Programação SBC! 🏆 A competição reuniu 26 times 🧑‍🎓, com participantes do INF-UFRGS, IFSul e UNISINOS.

📖 O grande destaque foi o time do INF-UFRGS "[UFRGS] Cacatinho implements FrenchBread", formado pelos nossos discentes Arthur Hofi, Leo Hardt e Matheus Mews, que conquistou o título de campeão da sede 🏆. A equipe contou com o treinamento do nosso ex-aluno Rodrigo Bervig e do professor Henrique Becker.

📖 O time campeão da UFRGS resolveu 7 problemas e, no ranking geral de todas as sedes do Brasil, ficou na 24ª posição. O melhor time da UNISINOS também resolveu 7 problemas, garantindo o segundo lugar da sede Porto Alegre e a 44ª posição no ranking geral — mostrando que a disputa foi equilibrada e emocionante 🏆🔥, decidida pelo tempo total de resolução dos problemas, critério de desempate da competição 🏆.

🏆 Com essa vitória, nossos estudantes garantem vaga na fase final brasileira, que acontecerá de 6 a 9 de novembro em São Paulo. A conquista reforça a tradição da UFRGS como uma das pouquíssimas universidades do país que já se classificou para todas as finais brasileiras da competição 🌟.

Tudo isso só foi possível pelo apoio das empresas: Appmax, Nelogica, Telus digital, Dell, Infracore, Int6Tech, widelabs, Aprix, MBOChip. A Appmax, em especial, foi responsável por um farto café da manhã para os competidores no sábado. 🍳

🏆 Parabéns aos campeões do INF-UFRGS! E para acompanhar mais sobre a Maratona, acesse o link na bio! 💙💚

21 sem

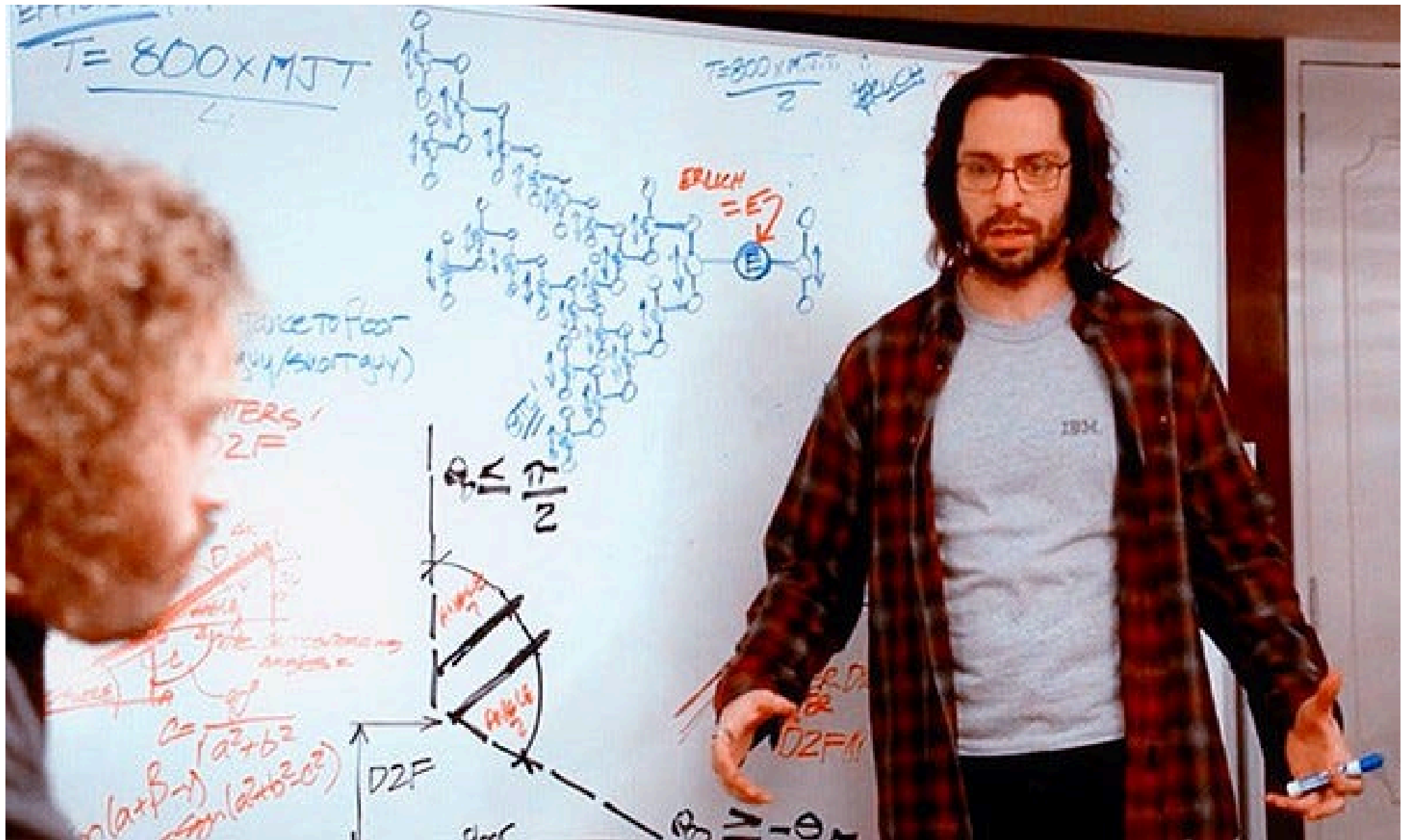
🤍💬🔖

👤 Curtido por dennis_giovani e outras 60 pessoas

17 de setembro de 2025

😊 Adicione um comentário... Postar

Motivação: Entrevista Técnica



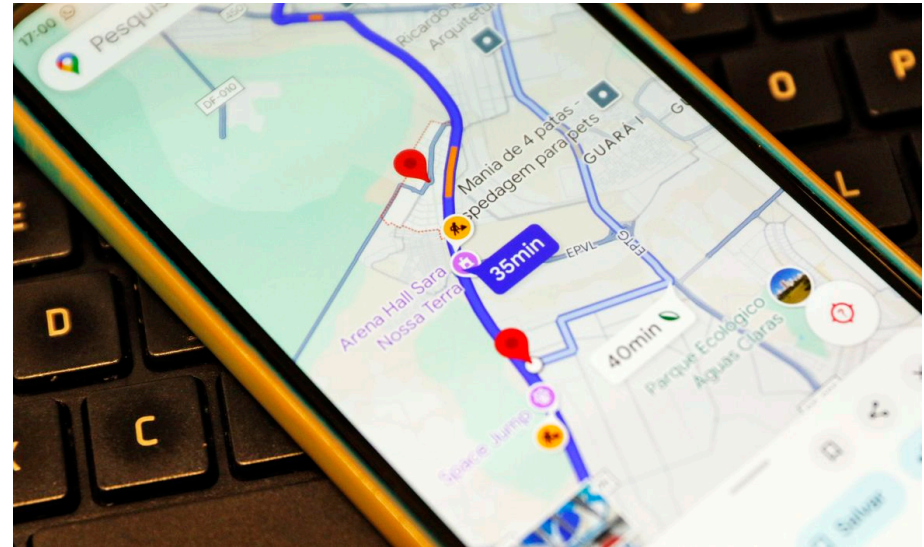
Silicon Valley

Motivação: Entrevista Técnica

E-mail Entrevista **Google**:

- Algorithm Complexity: If you struggle with basic big-O complexity analysis, then you are almost guaranteed not to get hired.
- Sorting: Know how to sort.
- Hashtables: Arguably the single most important data structure known to mankind.
- Trees: Know about trees; basic tree construction, traversal and manipulation algorithms.
- Graphs: Graphs are really important at Google.
- Other data structures: You should especially know about the most famous classes of NP-complete problems, . . . Find out what NP-complete means.
- Mathematics, Operating Systems, Coding.
- Final Tips: ***Get your algorithms straight.

Motivação: Resolver Problemas



ATGCC-CA-ATATAA-GGACCTGGCTCAAGGGAGACTGGAGTTCTAGGGGCTCTGGGAT-ATGTGGGGCGGTG-GG
--- CCTCAGA-A-AACTGATTAT C-CTTGGGA-A-GG-G--TC-A--- GTTCTCT--- TCAT-TAGGACAATGAAG
GACAGTGCAG-AGACCTTTTCACAGAGCCAAGGAGATAACCCA-GCACCCAGAGAGCAGAGC-AATCCACGGG-CT
AAAAATCCAGCTGTCC-CTTC-CA-AG--- GGGAGGTA--TCATG-A-GCAG-TATCA-AGGTAA-GCAAGGGAA-C-
CT-GTGTGGGAGTGAGGGAGGCGTTCCCGGCTTTTCATCCAGGTGCACCTGAGCCCGGATCCCCCATGAGTCTGTCT
CTGGT-TTGGAACTCCTGGTTGAC--ACCGGCTTTCACTTAAG--- A--- AG----- T--CGCAAGAGACTGCCG
CAGGA-AGTAAATGGCA-AAAGCGCTAAGTAGATAACCCAGAGGAGGAGGAATTCTG--- AATTCTGTT-TATAGC
CAAGAGAGGAA---GCCACAGA-C-C-AGGTTG-GA---CCCTGAG--CCTGGAG--TACTGCTTTTCTCTCT-TTGC
TCTGCT--CCTATTITG-TTTTCTA-A--CCTCAGA-A-A-A-CT----- GA----- TT--T-A-TC-CTTG--G-GA
-CTGCTAACT-CCTTGTCTGCAAGATTAGCAGAGATGACACCTTCTGGAAGGCTTCTGAGTCTCCTGAGTGA
----- AGGGTC--- AG--- TT--- TCTTCAT-TAGGA-CAATGAAGAAA--- A-ATC--CAGCTGTCCCTCCAAG--G
GCCCCTAGGCTCCCCTAGCCCTGGCTCTT-ATGTCTT ATCTCTG-AGAAATCTGAGATCTGCA-CT-TTGTAGAAAGTCC



Professor, mas e a IA?

Projeto e Análise de Algoritmos

Projeto: técnicas para construção

- Busca exaustiva (backtracking).
- Estratégia Gulosa (greedy).
- Divisão e Conquista. (*PAA2...*)
- Programação Dinâmica. (*PAA2...*)
- Modelagem e Otimização. (*Otimização Combinatória...*)

Análise: verificação da qualidade do algoritmo

- Correção
- Custo (em tempo)
- Custo (em espaço)

Algoritmos: processos computacionais

Problema do Troco (Coin Changing)

Problema do Troco (Coin Changing)

Objetivo. Dadas denominações de moedas (ex.: $\{1, 5, 10, 25, 100\}$), pagar um valor usando o **menor número de moedas**.



<https://brunogrisci.github.io/cashiers>

Problema do Troco (Coin Changing)

Objetivo. Dadas denominações de moedas (ex.: $\{1, 5, 10, 25, 100\}$), pagar um dado valor usando o **menor número de moedas**.

Algoritmo do caixa (guloso). Repetidamente escolha a **maior** moeda que **não excede** o valor restante.

Exemplos (ideia)

- 34: pegue 25, depois 5, depois 1,1,1,1.
- 2,89: escolha sempre a maior moeda possível e continue no restante.

Algoritmo do Caixa: Pseudocódigo

CASHIERSALGORITHM($x, c_1, \dots, c_n$)

```
1:  Ordene as moedas:  $0 < c_1 < c_2 < \dots < c_n$ 
2:   $S \leftarrow []$ 
3:  while  $x > 0$  do
4:     $k \leftarrow$  maior índice tal que  $c_k \leq x$ 
5:    if não existe tal  $k$  then
6:      return “sem solução”
7:    else
8:       $x \leftarrow x - c_k$ 
9:      adicione  $k$  ao final de  $S$ 
10: return  $S$ 
```

Escrevendo Pseudocódigo

- Não existe um formato universal rígido de pseudocódigo.
- O objetivo principal é a **clareza**: permitir que qualquer pessoa compreenda a lógica e possa implementá-la em qualquer linguagem.

Elementos Essenciais & Boas Práticas

- **Nome claro**: identificador descritivo para o algoritmo (ex.: `CashiersAlgorithm`).
- **Entrada (Input)**: quais são os dados que o algoritmo recebe e suas condições.
- **Saída (Output)**: o que o algoritmo produz ou retorna ao final.
- **Nível de abstração**: foco no fluxo lógico e controle, sem se prender a detalhes sintáticos de linguagens específicas.

O algoritmo do caixa é ótimo?

Pergunta. O algoritmo do caixa é ótimo para as moedas dos EUA $\{1, 5, 10, 25, 100\}$?

1. Sim, algoritmos gulosos são sempre ótimos.
2. Sim, para qualquer conjunto $c_1 < \dots < c_n$, desde que $c_1 = 1$.
3. Sim, por propriedades especiais das denominações dos EUA.
4. Não.

O algoritmo do caixa é ótimo?

Pergunta. O algoritmo do caixa é ótimo para as moedas dos EUA $\{1, 5, 10, 25, 100\}$?

1. Sim, algoritmos gulosos são sempre ótimos.
2. Sim, para qualquer conjunto $c_1 < \dots < c_n$, desde que $c_1 = 1$.
3. **Sim, por propriedades especiais das denominações dos EUA.**
4. Não.

Discussão: (A) é falsa em geral; (B) também é falsa; (C) é a correta nesse caso.

O algoritmo guloso é ótimo para qualquer sistema de moedas?

Resposta curta: **não**.

Contraexemplo (selo/“postage”): denominações 1, 10, 21, 34, 70, 100, 350, 1225, 1500.



- Guloso para 140: $100 + 34 + 1 + 1 + 1 + 1 + 1 + 1$ (8 moedas).
- Ótimo: $70 + 70$ (2 moedas).

Pode falhar até em viabilidade se $c_1 > 1$: denominações 7, 8, 9.

- Guloso para 15: escolhe 9 e fica preso (não completa).
- Ótimo: $7 + 8$.

Propriedades de uma solução ótima (moedas dos EUA)

Para denominações $\{1, 5, 10, 25, 100\}$, qualquer solução ótima satisfaz:

- **(P1)** número de pennies $P \leq 4$.
Prova por troca: substitua 5 pennies por 1 nickel.
- **(P2)** número de nickels $N \leq 1$.
- **(P3)** número de dimes $D \leq 2$.
- **(P4)** número de quarters $Q \leq 3$.
- **(P5)** $N + D \leq 2$.
Se $N = 1$ e $D = 2$, substitua por 1 quarter.

Consequência chave:

$$P + 5N + 10D + 25Q \leq 99.$$

(Isto limita o quanto dá para “fazer troco” sem usar moedas de 1 dólar.)

Otimidade do algoritmo do caixa (moedas dos EUA)

Teorema. O algoritmo do caixa é **ótimo** para $\{1, 5, 10, 25, 100\}$.

Ideia da prova (indução no valor x):

- Considere $c_k \leq x < c_{k+1}$. O guloso escolhe c_k .
- Mostra-se que **qualquer solução ótima** também precisa usar c_k : caso contrário, teria de compor x só com $c_1, \dots, c_{k-1}$, mas isso é impossível por limites como $P \leq 4, N \leq 1, N + D \leq 2, Q \leq 3$.
- Então o problema reduz a $x - c_k$, resolvido otimalmente pelo guloso (hipótese de indução).

Intuição: a prova é um “argumento de troca” + indução.

Solução ótima única (incluindo moedas de 1 dólar)

Proposição. O algoritmo do caixa produz a **única** solução ótima para $\{1, 5, 10, 25, 100\}$.

Esboço (caso das moedas de 1 dólar):

- Para um valor $x.yy$, o guloso escolhe x moedas de 1 dólar.
- Suponha, por contradição, que exista solução ótima com menos de x dólares.
- Então o restante em $\{1, 5, 10, 25\}$ precisa somar ≥ 100 , isto é, $P + 5N + 10D + 25Q \geq 100$, contradizendo o limite ≤ 99 .
- Argumentos análogos justificam as escolhas gulosas para quarters, dimes e nickels.

Para que conjuntos de moedas o algoritmo guloso é ótimo?

Teorema. Quando cada moeda é um **múltiplo inteiro da moeda imediatamente menor** e $c_1 = 1$, o algoritmo do caixa é **ótimo**.

Prova. Suponha um sistema de moedas $c_1, c_2, \dots, c_n$. onde $c_1 = 1$ e cada moeda $c_{i+1} = c_i \cdot k_i$ para algum inteiro $k_i \geq 2$.

- Logo, nenhuma solução ótima pode usar $\geq k_i$ moedas de cada tipo c_i , pois, se usasse, poderíamos trocar k_i moedas de tipo c_i por uma única moeda de tipo c_{i+1} , o que resultaria em um número menor de moedas, contradizendo a suposição de que a solução é ótima.
- Portanto, em uma solução ótima, a quantidade máxima permitida de qualquer moeda c_i é sempre $k_i - 1$.
- Essa é uma condição **suficiente**, não necessária, para que o algoritmo guloso seja ótimo. Não existe uma única propriedade matemática simples e fechada que defina todos os sistemas de moedas canônicos.

Conjuntos de Moedas Não Canônicos e Programação Dinâmica

- Em 2005, David Pearson publicou um algoritmo que consegue verificar se qualquer conjunto de moedas é canônico em tempo polinomial $O(n^3)$.
- Para sistemas de moedas não canônicos, existe algoritmo ótimo que utiliza programação dinâmica (PAA II):

Algoritmo de Programação Dinâmica

Implementação do Algoritmo do Caixa (Python)

Implementação do algoritmo do caixa (Python)

Github: Implementação

Python Code [↺ Start Over](#) [▶ Run Code](#)

```
1
2 from typing import List
3
4 def greedy_troco(valor: float, moedas: List[int]):
5     """Algoritmo guloso para o problema do troco."""
6     return {}
7
8
9 moedas = [1, 5, 10, 25, 50, 100]
10
11 valor = 13.42
12 troco = greedy_troco(valor, moedas)
13 print(troco)
```

Implementação do algoritmo do caixa (Python)

Github: Implementação

Implementação do algoritmo do caixa (Python)

↺ Start Over

▶ Run Code

```
1  from typing import List
2
3  def greedy_troco(valor: float, moedas: List[int]):
4      """Algoritmo guloso para o problema do troco."""
5      moedas = sorted(moedas, reverse=True)
6      troco = {m : 0 for m in moedas}
7      valor *= 100
8      while valor != 0:
9          sem_solucao = True
10         for m in moedas:
11             if m <= valor:
12                 troco[m] += 1
13                 valor -= m
14                 sem_solucao = False
15                 break
16         if sem_solucao:
17             print("Sem solução!")
18             return troco
19     return troco
20
21
22 moedas = [1, 5, 10, 25, 50, 100]
```

```
23  
24 valor = 13.42  
25 troco = greedy_troco(valor, moedas)
```

Concluindo

- Neste primeiro mês, vamos abordar a base para a **análise de correção e de custos de algoritmos**, em particular a **notação assintótica**.
- Daqui para a frente, a ideia é saber desenvolver algoritmos tendo uma compreensão profunda sobre:
 - **por que ele funciona (corretude)** e
 - seu respectivo **custo** (em tempo e espaço).

